WIF3009
Python for Scientific Computing

Introduction to Python (Part 2)

Hazim Hanif

Numpy

- Recall that regular Python lists do not support element wise operations.

```
[202]: heights = [1.24, 1.67, 1.98]
       weights = [45, 56, 80]

       weights * heights

       ---------------------------------------------------------------------------
       TypeError                                 Traceback (most recent call last)
       Cell In[202], line 4
             1 heights = [1.24, 1.67, 1.98]
             2 weights = [45, 56, 80]
       ----> 4 weights * heights

       TypeError: can't multiply sequence by non-int of type 'list'
```

- This limitation is critical as most operations in data science require element wise procedure to speed up the process.
- The solution to this is the NumPy package.

Numpy

- NumPy is short for Numeric Python.
- It provides an alternative to Python List through its NumPy Array.
- The structure is similar to the regular list but it supports calculations over the entire array.
- It is easy and operates very fast.
- To use NumPy package, you need to install it as it does not come preinstalled. There are multiple ways to install. E.g:
  - `pip3 install numpy`
  - `conda install numpy`

Numpy

- Once you have installed NumPy, you need to `import` the package before it becomes available in your Python environment.
- The import requirement remains true for all external packages that you wish to use.
- The import statement should be at the top of your script as follows:

```
[208]: import numpy as np

[209]: heights = [1.24, 1.67, 1.98]
       weights = [45, 56, 80]

[210]: np_heights = np.array(heights)
       np_heights

[210]: array([1.24, 1.67, 1.98])

[211]: np_weights = np.array(weights)
       np_weights

[211]: array([45, 56, 80])
```

Numpy

- Example of element-wise division/multiplication

```
[214]: heights = [1.24, 1.67, 1.98]
       weights = [45, 56, 80]

[215]: np_weights/np_heights

[215]: array([36.29032258, 33.53293413, 40.4040404 ])

[216]: np_weights*2

[216]: array([ 90, 112, 160])
```

Numpy

- The figure from previous slide showed that the two original Python Lists have been converted into NumPy Arrays.
- As a result, you can easily calculate the BMIs for the entire array.
- Unlike Python List which can contain multiple data types in a single list, NumPy array contain only one type. If you input more than one type, Python will forcefully convert (type coercion) into one single type i.e. object/string
- Also, Python List and NumPy array demonstrate different behaviors even when acted on a same operation.

Numpy

- A regular Python list gets appended with the plus (+) sign.
- A NumPy array gets added element wise with the plus (+) sign.

```
[217]: python_list = [1,2,3]
       python_list

[217]: [1, 2, 3]

[219]: numpy_list = np.array([1,2,3])
       numpy_list

[219]: array([1, 2, 3])

[220]: python_list + python_list

[220]: [1, 2, 3, 1, 2, 3]

[221]: numpy_list + numpy_list

[221]: array([2, 4, 6])
```

Numpy (subsetting)

- Subsetting a NumPy array is the same as you have done with Python list: using the square brackets [ ]. E.g. `np_weights[2]`
- However, NumPy array also accepts Boolean condition/comparison to subset its array.
- For example, to find weights above 50, you can do the following:
  - `np_weights > 50`
  - `np_weights[np_weights > 50]`
- The first option returns an array of Boolean while the second option returns an array of values that meet the Boolean condition.

Numpy (subsetting)

- Example from the previous slide
- Subsetting a numpy array based on a condition.

```
[224]: np_weights
[224]: array([45, 56, 80])
[227]: np_weights[1]
[227]: 56
[225]: np_weights > 50
[225]: array([False,  True,  True])
[226]: np_weights[np_weights > 50]
[226]: array([56, 80])
```

Numpy (dimensionable array)

- The NumPy array that we have learned until now is 1-dimensional array. If you check the type(np_heights), it will return numpy.ndarray.
- ndarray means N-dimensional array. In other words, NumPy is able to hold more than 1-dimensional array.
- For example, to create a 2D array, you can do the following:

```
[228]: np_2d = np.array([[2.3,66.5,34.2],
                         [12.4,23.2,33.5]])

[229]: np_2d

[229]: array([[ 2.3, 66.5, 34.2],
              [12.4, 23.2, 33.5]])

[230]: np_2d.shape

[230]: (2, 3)
```

Numpy (subsetting)

- Subsetting a 2D NumPy array is similar to subsetting Python list containing lists. It needs double brackets. But, there are a few other options as follows, depending on the preferred final output:
  - `np_2d[0]` : returns the first row, all columns
  - `np_2d[0]`[2] : returns the first row, third column
  - `np_2d[0,2]` : returns the first row, third column
  - `np_2d[0,` [0,2]] : returns the first row, first and third column
  - `np_2d[:, 1:3]` : returns all rows, second and third column (column index 3 not inclusive).

Numpy (boolean comparison)

- As briefly mentioned, you can subset/filter your NumPy array using Boolean/Comparators e.g. `np_heights[np_heights > 1.6]]`
- There are other Boolean operators such as `and`, `or` and `not`.
  - `and` operator returns True if both conditions are True.
  - `or` operator returns True if any of the conditions are True.
  - `not` operator returns the reverse of the Boolean output.

Numpy (boolean comparison)

- However, doing `np_weights[np_weights > 50 and np_weights < 80]` yield an error.
- Using boolean operators with numpy array.

```
[237]: np_weights

[237]: array([45, 56, 80])

[241]: np_weights[(np_weights <60) & (np_weights>50)]

[241]: array([56])

[242]: np_weights[(np_weights < 50) | (np_weights > 60)]

[242]: array([45, 80])
```

User Input

- Python allows for user input.
- That means we are able to ask the user for input.
- We can use the *input( )*.

```
[243]: username = input("Enter your name:")
       print("Your name is: " + username)

       Enter your name: hazim
       Your name is: hazim
```

Iterators

- An iterator is an object that contains a countable number of values.
- An iterator is an object that can be iterated upon, meaning that you can traverse through all the values.
- Technically, in Python, an iterator is an object which implements the iterator protocol, which consist of the methods **__iter__()** and **__next__().**
- Lists, tuples, dictionaries, and sets are all iterable objects.
    - All these objects have a iter() method which is used to get an iterator:

Iterators

- Using the iterator on a tuple.

```
[269]: mytuple = ("chicken", "beef", "egg")
       myit = iter(mytuple)

       print(next(myit))
       print(next(myit))
       print(next(myit))

       chicken
       beef
       egg
```

Function

- A **function** is a block of code which only runs when it is called.
- You can pass data, known as parameters, into a function.
- A function can return data as a result.
- We can use the ‘def’ keyword

```
[244]: def my_func():
           print('This is inside function')

[245]: my_func()

       This is inside function
```

Function

- A function that accept one argument.

```
[247]: def my_func(x):
           myvar = x * 2
           print(myvar)

[249]: my_func(12)

       24
```

- A function that accept multiple arguments.

```
[262]: def my_func(x, y, z):
           myvar = x + y + z
           return myvar

[263]: retVal = my_func(2,3,4)
       print(retVal)

       9
```

Lambda

- A lambda function is a small anonymous function.
- A lambda function can take any number of arguments, but can only have one expression.
- The power of lambda is better shown when you use them as an anonymous function inside another function.

Syntax

```
lambda arguments : expression
```

Lambda

- Lambda function with one argument, one expression.

```
[265]: val = lambda x: x + 2

[266]: val(2)

[266]: 4
```

- Lambda function multiple arguments, one expression.

```
[267]: val = lambda x,y,z: x+y+z

[268]: val(2,3,4)

[268]: 9
```

Scope (local)

- A variable is only available from inside the region it is created.
- This is called **scope**.
- A variable created inside a function belongs to the ***local scope*** of that function, and can only be used inside that function.

```
[274]: x = 10

       def myfunction():
         x = 30
         print(x)

[275]: myfunction()

       30

[276]: print(x)

       10
```

Scope (local)

- A variable not available outside the function, but it is available for any function inside the function.

```
[283]: x = 10

       def myfunction():
         x = 30

         def myinnerfunction():
           print(x)

         return myinnerfunction()

[284]: myfunction()

       30

[285]: print(x)

       10
```

Scope (global)

- A variable created in the main body of the Python code is a global variable and belongs to the global scope.
- Global variables are available from within any scope, global and local.

```
[288]: x = 10

       def myfunction():
         print(x)

[289]: myfunction()

       10
```

Scope (global)

- Manipulating the global scope/variables.

```
[290]: x = 10

       def myfunction():
           x = 3
           print(x)

[291]: myfunction()

       3

[292]: print(x)

       10
```

NOT WORKING

```
[293]: x = 10

       def myfunction():
           global x
           x = 3
           print(x)

[294]: myfunction()

       3

[295]: print(x)

       3
```

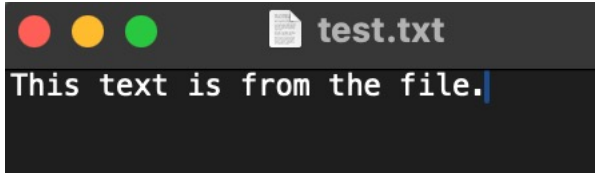
WORKING

File Handling

- Python has several functions for creating, reading and updating files.
- The key function for working with files in Python is the open() function.
- The open() function takes two parameters; *filename*, and *mode*.
- There are four different methods (modes) for opening a file:
  - "r" - **Read** - Default value. Opens a file for reading, error if the file does not exist
  - "a" - **Append** - Opens a file for appending, creates the file if it does not exist
  - "w" - **Write** - Opens a file for writing, creates the file if it does not exist
  - "x" - **Create** - Creates the specified file, returns an error if the file exist
- In addition you can specify if the file should be handled as binary or text mode
  - "t" - Text - Default value. Text mode
  - "b" - Binary - Binary mode (e.g. images)

File Handling (reading)

- Reading a text file.



Text file (test.txt)

```
[305]: reader = open('test.txt','r')

[306]: reader.readlines()

[306]: ['This text is from the file.'

[316]: reader.close()
```

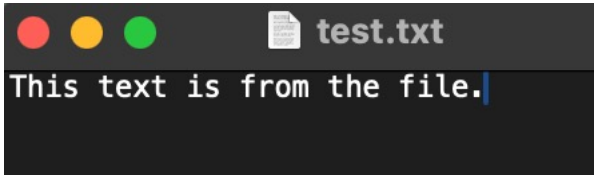
File Handling (writing)

- Writing to a text file, overwriting everything it in.
- Using mode 'w'

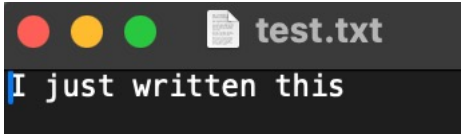
```
[307]: writer = open('test.txt','w')

[308]: writer.writelines('I just written this')

[309]: writer.close()
```



Before



After (overwritten)

File Handling (writing)

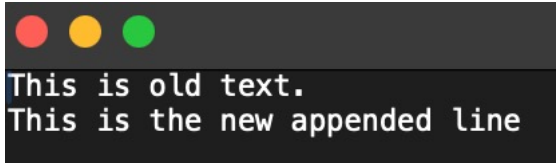
- Writing to a text file, but this time appending the existing texts.
- Using mode 'a'

```
[313]: writer = open('test.txt','a')

[314]: writer.writelines('\nThis is the new appended line')

[315]: writer.close()
```

Before



After

Classes

- Python is an object-oriented programming language.
- Almost everything in Python is an object, with its properties and methods.
- A Class is like an object constructor, or a "**blueprint**" for creating objects
- To create a class, use the keyword class:

```
[320]: class MyClass:
           x = 5

[321]: obj1 = MyClass()

[324]: obj1.x

[324]: 5
```

Classes (constructor)

- To understand the meaning of classes we have to understand the built-in **__init__()** function.
- All classes have a function called **__init__()**, which is always executed when the class is being initiated.
- Use the **__init__()** function to assign values to object properties when the object is being created.

```
[325]: class Person:
           def __init__(self, name, age):
               self.name = name
               self.age = age

[326]: student1 = Person("John",23)

[327]: student1.name

[327]: 'John'

[328]: student1.age

[328]: 23
```

Classes (methods)

- Objects can also contain methods. Methods in objects are functions that belong to the object.
- Let us create a method in the Person class.

```
[334]: class Person:
           def __init__(self, name, age):
               self.name = name
               self.age = age

           def speak(self):
               print('I am {}'.format(self.name))

[335]: student1 = Person("John",23)

[336]: student1.speak()

       I am John
```

Classes (mutating)

- **Changing** the value of an object properties.
- You don't need an accessor or mutator method since it is public by default.

```
[350]: class Person:
           def __init__(self, name, age):
               self.name = name
               self.age = age

           def speak(self):
               print('I am {}'.format(self.name))

[351]: student1 = Person("John",23)

[352]: student1.age

[352]: 23

[353]: student1.age = 40

[354]: student1.age

[354]: 40
```

Classes (mutating)

- **Deleting** the value of an object properties.

```
[367]: class Person:
           def __init__(self, name, age):
               self.name = name
               self.age = age

           def speak(self):
               print('I am {}'.format(self.name))

[368]: student1 = Person("John",23)

[369]: student1.age

[369]: 23

[370]: del student1.age

[371]: student1.age

---------------------------------------------------------------------------
AttributeError                            Traceback (m
Cell In[371], line 1
----> 1 student1.age

AttributeError: 'Person' object has no attribute 'age'
```

Exception Handling

- The **try** block lets you test a block of code for errors.
- The **except** block lets you handle the error.
- The **else** block lets you execute code when there is no error.
- The **finally** block lets you execute code, regardless of the result of the try- and except blocks.

Exception Handling

- Try and Except

Without exception handling (crashed) →

```
[374]: print(non_existent_variable)

---------------------------------------------------------------
NameError                                 Traceback (mo
Cell In[374], line 1
----> 1 print(non_existent_variable)

NameError: name 'non_existent_variable' is not defined
```

With exception handling (no crash) →

```
[373]: try:
         print(non_existent_variable)
       except:
         print("An exception occurred")

       An exception occurred
```

Exception Handling (multiple exceptions)

- You can define as many exception blocks as you want,
- Example: if you want to execute a special block of code for a special kind of error:

```
[375]: try:
           print(non_existent_variable)
       except NameError:
           print('This is a nameerror exception')
       except:
           print('This is other exception')

       This is a nameerror exception
```

Exception Handling (finally)

- Finally will always be executed regardless the error is captured in any of the exeception handler.

```
[377]: try:
           print(non_existent_variable)
       except NameError:
           print('This is a nameerror exception')
       except:
           print('This is other exception')
       finally:
           print('Will always execute this')

       This is a nameerror exception
       Will always execute this
```

Exception Handling (tips/tricks)

- Sometimes, in a certain situation, we just want to catch the exception, but still continue with the execution of the code.
- You can easily achieve that with a simple try-except:

```
[383]: try:
           print(non_existent_variable)
       except:
           pass

       print('Continue...')

       Continue...
```

Tips & Tricks for Loops

- There are some statement that we can use to move our way around a loop (for, while, etc).
- Might be useful in the future
- These statements are: *continue*, *break*, *pass*
- *continue* -> skips the execution of the current iteration in the loop
- *break -> break out of the loop*

Tips & Tricks for Loops (continue)

- What if we want to do something based on a certain condition
- Here we want to skip the current iteration and proceed with the next one.

A normal looking FOR loop

```
[385]: fruits = ["apple", "banana", "cherry"]

[386]: for fruit in fruits:
           print(fruit)

       apple
       banana
       cherry
```

Added a condition and continue statement

```
[389]: fruits = ["apple", "banana", "cherry"]
       for fruit in fruits:
           if fruit == 'banana':
               continue
           print(fruit)

       apple
       cherry
```

Tips & Tricks for Loops (break)

- What if we want to do something based on a certain condition
- Here we want to break out of the loop.

A normal looking FOR loop

```
[385]: fruits = ["apple", "banana", "cherry"]

[386]: for fruit in fruits:
           print(fruit)

       apple
       banana
       cherry
```

Added condition and break statement

```
[390]: fruits = ["apple", "banana", "cherry"]
       for fruit in fruits:
           if fruit == 'banana':
               break
           print(fruit)

       apple
```